NEXT GEN SOFTWARE HUB

CYBER SECURITY INTERNSHIP

WEEK 07 TASK 07 REPORT

OWASP Top 10 – Part 1 (Injection, XSS, CSRF)

Understanding and Exploiting Common Web Vulnerabilities

TOPICS

- Injection (SQLi)
- XSS
- CSRF

TOOLS

- DVWA
- Burp Suite
- OWASP ZAP

RESOURCES

- OWASP Top 10 Official Site
- DVWA Setup Guide

TASK

Exploit SQLi and XSS on DVWA and document results.

Week 7: OWASP Top 10 – Part 1 (Injection, XSS, CSRF)

PLANS :

Week 7: OWASP Top 10 – Part 1 (Injection, XSS, CSRF)



PREPARED BY

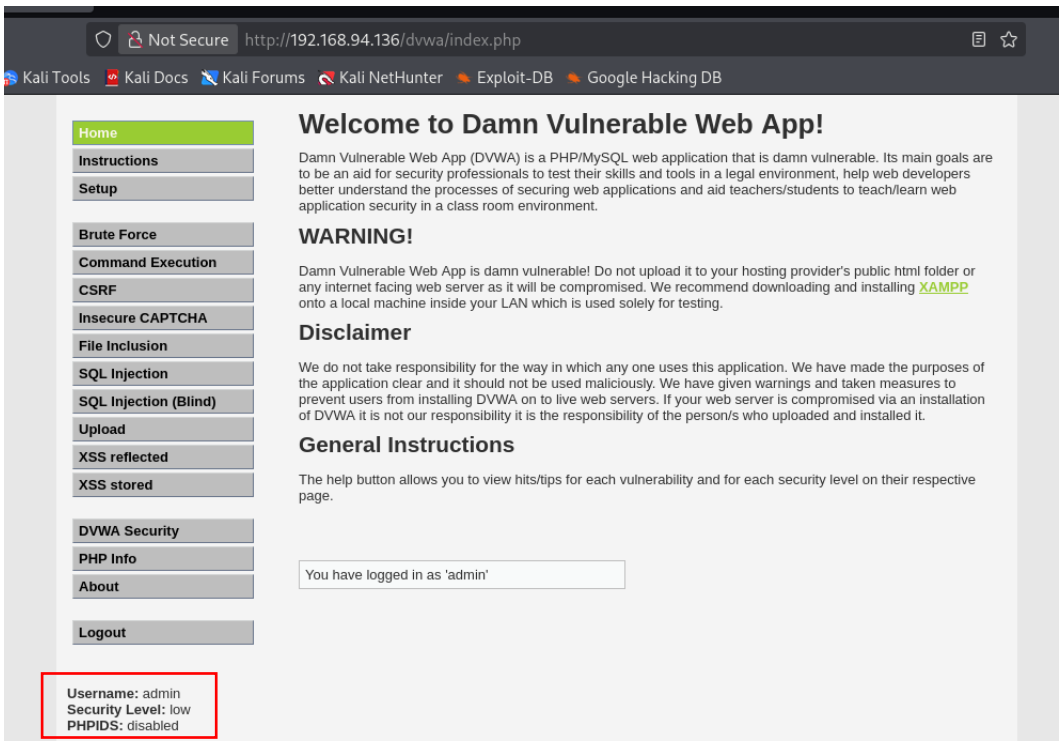
Selo David Sethibo

Cyber Security Intern

2026

Introduction and Objective

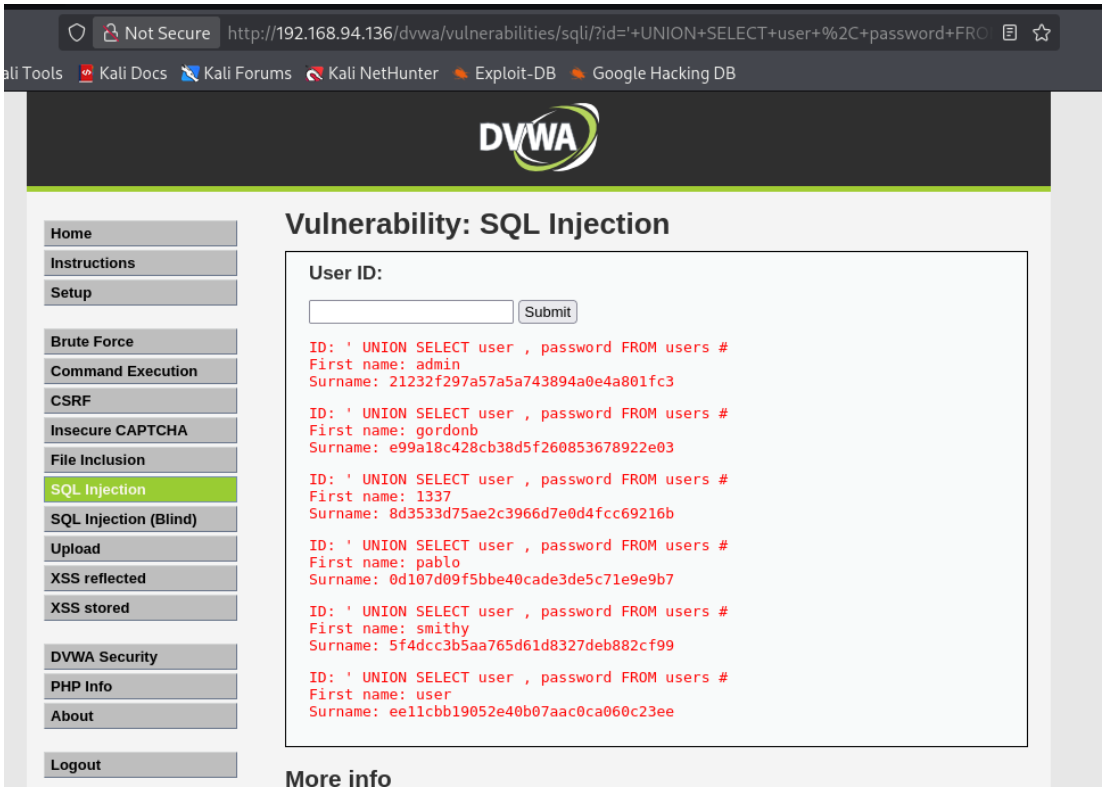
The objective of this assessment was to explore, exploit, and document three of the most critical vulnerabilities listed in the OWASP Top 10: SQL Injection, Cross-Site Scripting, and Cross-Site Request Forgery. For this practical, the Damn Vulnerable Web Application (DVWA) hosted on the OWASP Broken Web Applications virtual machine was utilized at the IP address 192.168.94.136. The security level of the application was deliberately set to Low to simulate a system with no defensive controls. This configuration allowed the assessor to understand the raw mechanics of how these attacks function, how they are executed, and the severe impact they can have on an organization's data and users.



Phase One: SQL Injection

SQL Injection occurs when an application fails to properly sanitize or validate user input before passing it to a backend database. This vulnerability is exceptionally dangerous in the real world, as it often leads to massive data breaches where millions of customer records, credit card numbers, or proprietary business data are stolen. The testing began by interacting with the User ID search field. By entering a crafted payload consisting of a single quote to close the existing database query, followed by a UNION SELECT statement, the assessor was able to append a malicious query to the original one.

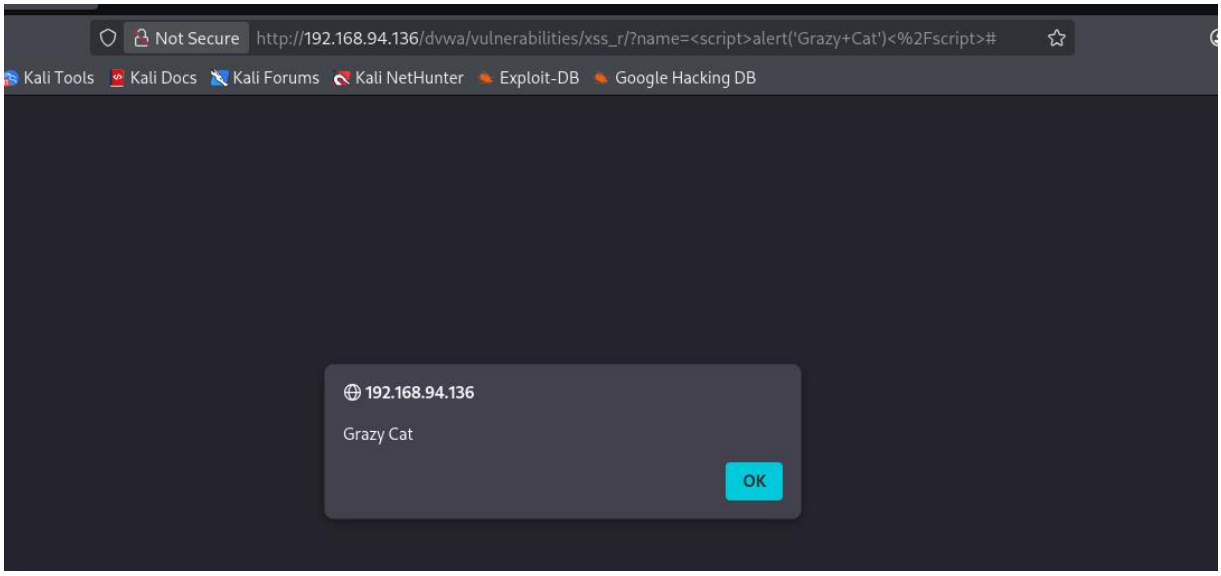
The specific payload instructed the database to retrieve the user and password columns from the users table. The application executed this injected query flawlessly. The resulting web page displayed a complete list of all registered database users, including the administrative account, alongside their corresponding password hashes. This demonstrated a total lack of input validation and parameterized queries within the application logic. In a real-world scenario, an attacker would then take these hashes and attempt to crack them offline to gain full administrative access to the system.



Phase Two: Cross-Site Scripting (XSS)

Cross-Site Scripting allows an attacker to inject malicious client-side scripts into web pages viewed by other users. This vulnerability is frequently used in real-world attacks to steal session cookies, allowing an attacker to hijack a victim's active login session without ever needing their password. It can also be used to redirect users to phishing pages or to perform actions on behalf of the victim.

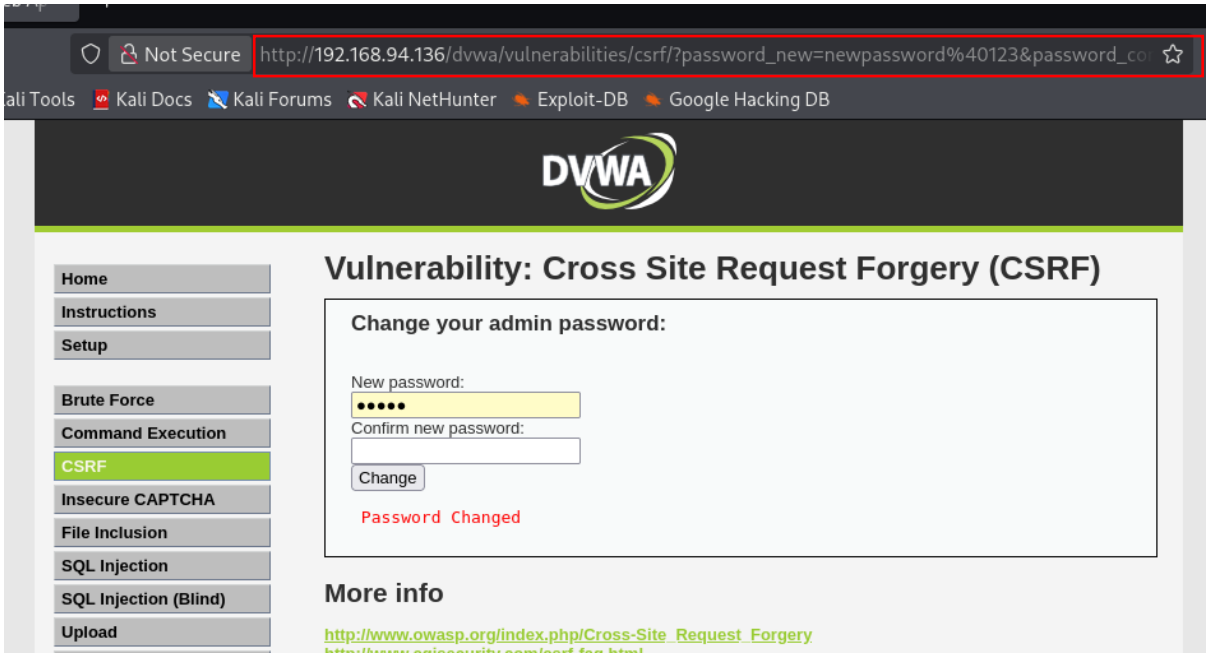
The assessment focused on Reflected XSS, which occurs when the injected script is returned immediately by the server and executed by the victim's browser. A script payload designed to trigger a browser alert was submitted into the name input field. The specific payload used was a JavaScript alert function. The application executed the code successfully, proving that the server reflected the input without any sanitization or context-aware output encoding. This confirms that an attacker could craft a malicious link containing this script and send it to a victim, leading to complete client-side compromise once the link is clicked.



Phase Three: Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery is an attack that forces an authenticated user to execute unwanted actions on a web application in which they are currently authenticated. In the real world, this is often used to trick users into changing their email addresses, transferring funds, or changing their passwords, all without their knowledge.

On the password change page within DVWA, it was observed that the application did not require the user to enter their current password, nor did it implement a unique, unpredictable anti-CSRF token. A new password was entered, and the resulting action was submitted. By inspecting the resulting URL, it became evident that the entire password change function was handled via a simple GET request containing the new password in plain text within the URL parameters. Because the request lacked a token to verify the origin of the request, an attacker could easily craft a malicious link and send it to a logged-in user. If the user clicks the link, their password is silently changed without their consent, effectively locking them out of their own account.



Conclusion

This practical assessment highlighted the devastating impact of missing input validation, improper output encoding, and poor session management. SQL Injection allows for complete database compromise and data theft, XSS allows for session hijacking and client-side manipulation, and CSRF allows for unauthorized state changes. The key takeaway is that modern applications must never trust user input. Implementing prepared statements for database queries, utilizing context-aware output encoding for web pages, and enforcing the use of unpredictable anti-CSRF tokens are mandatory practices for modern web application security.